



ICET Certified Master

COURSEWORK

Module	Internet Technology
Coursework Title	Project API
Coursework Type	Final Course Work

Table of Contents

1. Introduction & Overview	3
2. Authentication	4
3. API Response Format	5
4. Bank Loan Management API	6
5. Bus Booking API	9
6. College Project API	13
7. E-Commerce API	16
8. Employee App API	21
9. Employee Onboarding API	23
10. Enquiry Management API	27
11. Fees Tracking API	29
12. Goal Tracker API	32
13. Leave Tracker API	36
14. Project Competition API	39
15. Smart Parking API	42
16. Survey API	47
17. User App API	51
18. Error Handling & HTTP Status Codes	53
19. Integration Guide for HTML/JS/Bootstrap	54

1. Introduction & Overview

This document provides a comprehensive reference for the FreeProject REST API, a multi-application backend built with ASP.NET Core following the OpenAPI 3.0.1 specification. The API serves as the backend for twelve distinct application modules, each targeting a different domain of functionality.

1.1 Supported Applications

Module	Controller	Purpose
Bank Loan	/api/BankLoan	Loan application submission, customer & bank employee management
Bus Booking	/api/BusBooking	Bus vendor management, route/schedule search, seat booking
College Project	/api/CollegeProject	Student project submissions and status tracking
E-Commerce	/api/Ecommerce	Product catalog, cart, orders, customers and addresses
Employee App	/api/EmployeeApp	Basic employee CRUD with department/designation lookups
Employee Onboarding	/api/EmployeeOnboarding	Full employee lifecycle, master data, and company management
Enquiry Management	/api/Enquiry	Customer enquiry tracking with categories and statuses
Fees Tracking	/api/FeesTracking	Student enrollment, fee payments, and batch management
Goal Tracker	/api/GoalTracker	Personal goal-setting, tasks, milestones, and reminders
Leave Tracker	/api/LeaveTracker	Employee leave requests and balance management
Project Competition	/api/ProjectCompetition	Competition creation, project submissions, and judging
Smart Parking	/api/SmartParking	Multi-site parking management, slot tracking, and reporting
Survey	/api/Survey	Survey creation, question management, and response collection
User App	/api/UserApp	JWT-based authentication, user management, password reset

1.2 Frontend Technology Stack

Frontend: HTML5 + Vanilla JavaScript (ES6+) + Bootstrap 5

Base URL: <https://your-api-domain.com> (replace with actual deployment URL)

Content-Type: All POST/PUT request bodies must include Content-Type: application/json

2. Authentication

Most endpoints use simple credential-based login. The User App module additionally supports JWT (JSON Web Tokens) with refresh token functionality for more secure applications.

2.1 Standard Login (Most Modules)

Send a POST request with username/password credentials. On success the API returns user data including a role identifier.

Endpoint: POST /api/{ModuleName}/login

Field	Type	Required	Description
userName / emailId	string	Yes	User's username or email address
password	string	Yes	User's password (plain text)

2.2 JWT Authentication (User App)

The UserApp module returns a JWT access token and a refresh token. Store both in sessionStorage and include the Bearer token in subsequent requests.

Field	Type	Required	Description
emailId	string	Yes	Registered email address
password	string	Yes	Account password

Example JavaScript fetch with Bearer token:

```
const res = await fetch('/api/UserApp/login', { method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({emailId, password}) });
```

3. API Response Format

Many endpoints return a standard ApiResponse wrapper object. Understanding this structure is essential for consistent error handling in your frontend JavaScript.

Field	Type	Required	Description
message	string	No	Human-readable message describing the result
result	boolean	Yes	true on success, false on failure
data	any	No	The actual payload — could be an object, array, or null

Tip: Always check `response.result === true` before using `response.data` in your JavaScript code.

4. Bank Loan Management API

The Bank Loan module manages loan applicants, applications, and bank employee workflows. Customers submit applications; bank employees review and process them.

4.1 Endpoints

Method	Endpoint	Description
GET	/api/BankLoan/GetAllUsers	Retrieve all registered users
POST	/api/BankLoan/login	Authenticate a user (customer or bank employee)
POST	/api/BankLoan/RegisterCustomer	Register a new customer account
POST	/api/BankLoan/RegisterAsBankUser	Register a bank employee account
POST	/api/BankLoan/UpdateUser	Update existing user details
DELETE	/api/BankLoan/DeleteUserByUserId	Delete a user by userId (query param)
GET	/api/BankLoan/GetAllApplications	Get all loan applications (admin/bank employee)
GET	/api/BankLoan/GetMyApplications	Get applications for a specific customer (customerId)
GET	/api/BankLoan/GetApplicationAssignedToMe	Get applications assigned to a bank employee (bankEmployeeId)
GET	/api/BankLoan/CheckApplicationStatus	Check status by PAN card number and status string
POST	/api/BankLoan/AddNewApplication	Submit a new loan application

4.2 Key Request Models

LoginUser

Field	Type	Required	Description
userName	string	No	Username for login
password	string	No	Password for login

ApplicantLoanView — Submit a Loan Application

Field	Type	Required	Description
fullName	string	No	Full legal name of the applicant
panCard	string	No	PAN card number (unique identifier)
dateOfBirth	date-time	No	Applicant's date of birth
email	string	No	Contact email address
phone	string	No	Contact phone number
address / city / state / zipCode	string	No	Residential address details
annualIncome	double	No	Annual income in currency

employmentStatus	string	No	e.g. Employed, Self-Employed, Unemployed
creditScore	integer	No	Credit score (numeric)
assets	string	No	Description of assets held
loans	ApplicantsLoan[]	No	Array of existing loan details
customerId	integer	No	Link to the registered customer account

5. Bus Booking API

A full-featured bus ticketing system. Passengers search for routes, view available schedules, select seats and confirm bookings. Vendors manage their bus fleet and schedules.

5.1 Endpoints

Method	Endpoint	Description
GET	/api/BusBooking/GetAllUsers	Get all registered users
POST	/api/BusBooking/login	User login
POST	/api/BusBooking/AddNewUser	Register a new user
POST	/api/BusBooking/UpdateUser	Update user details
DELETE	/api/BusBooking/DeleteUserByUserId	Delete user by userId
GET	/api/BusBooking/GetBusVendors	List all bus vendors
GET	/api/BusBooking/GetBusVendorsById	Get a single vendor by id
POST	/api/BusBooking/PostBusVendor	Create a new bus vendor
PUT	/api/BusBooking/PutBusVendors	Update a bus vendor by id (query param)
DELETE	/api/BusBooking/DeleteBusVendor	Delete a bus vendor by id
GET	/api/BusBooking/GetBusLocations	Get all bus stop locations
GET	/api/BusBooking/GetBusLocationById	Get a single location by id
GET	/api/BusBooking/getAddressByLocationId	Get stop addresses for a location
POST	/api/BusBooking/PostBusLocationAddress	Add a new address point to a location
POST	/api/BusBooking/PostBusLocation	Create a new bus location
PUT	/api/BusBooking/PutBusLocation	Update a bus location by id
DELETE	/api/BusBooking/DeleteBusLocation	Delete a bus location
GET	/api/BusBooking/GetAvailableRoutes	List all available origin-destination route combinations
GET	/api/BusBooking/GetBusSchedules	Get all schedules for a vendor (vendorId query param)
GET	/api/BusBooking/searchBus	Search buses by fromLocation, toLocation, travelDate
GET	/api/BusBooking/getBookedSeats	Get list of booked seat numbers for a schedule
GET	/api/BusBooking/GetBusScheduleById	Get a single schedule by id
POST	/api/BusBooking/PostBusSchedule	Create a new bus schedule
PUT	/api/BusBooking/PutBusSchedule	Update a bus schedule
DELETE	/api/BusBooking/DeleteBusSchedule	Delete a bus schedule
GET	/api/BusBooking/GetAllBusBookings	Get all bookings for a vendor
GET	/api/BusBooking/GetBusBooking	Get a single booking by id
POST	/api/BusBooking/PostBusBooking	Create a new booking with passenger details
DELETE	/api/BusBooking/DeleteBusBooking	Cancel/delete a booking

5.2 Key Request Models

BusSchedule

Field	Type	Required	Description
vendorId	integer	No	ID of the bus vendor operating this schedule
busName	string	No	Display name of the bus
busVehicleNo	string	No	Vehicle registration number
fromLocation / toLocation	integer	No	Origin and destination location IDs
departureTime / arrivalTime	date-time	No	Departure and arrival date-times
scheduleDate	date-time	No	Date of travel
price	double	No	Ticket price per seat
totalSeats	integer	No	Total seating capacity

BusBookingViewModel — Create Booking

Field	Type	Required	Description
custId	integer	No	Customer ID making the booking
bookingDate	date-time	No	Date of booking
scheduleId	integer	No	ID of the selected bus schedule
busBookingPassengers	BusBookingPassenger[]	No	Array of passenger details

BusBookingPassenger

Field	Type	Required	Description
passengerName	string	No	Full name of passenger
age	integer	No	Passenger age
gender	string	No	Passenger gender
seatNo	integer	No	Selected seat number

6. College Project API

Manages student project submissions for academic review. Students submit projects with technical details; supervisors review and update statuses.

6.1 Endpoints

Method	Endpoint	Description
GET	/api/CollegeProject/GetAllUsers	Get all registered users
POST	/api/CollegeProject/login	User login
POST	/api/CollegeProject/AddNewUser	Register a new user
GET	/api/CollegeProject/getAllProjects	Retrieve all submitted projects
GET	/api/CollegeProject/getProjectByStatus	Filter projects by status string
GET	/api/CollegeProject/getProjectByUser	Get projects submitted by a specific user (userid)
GET	/api/CollegeProject/getDashboard	Get dashboard statistics (total, active, drafts)
GET	/api/CollegeProject/{id}	Get a single project by ID
PUT	/api/CollegeProject/{id}	Update an existing project
DELETE	/api/CollegeProject/{id}	Delete a project by ID
POST	/api/CollegeProject/SubmitProject	Submit a new project
GET	/api/CollegeProject/changeProjectStatus	Change a project's status (projectId + status query params)

6.2 ProjectSubmission Model

Field	Type	Required	Description
studentName	string	Yes	Full name of the student (max 255 chars)
contactNo	string	Yes	10-digit contact number
emailId	string	Yes	Valid email address
status	string	No	Current status (e.g. Draft, Active, Completed)
projectTitle	string	Yes	Title of the project (max 255 chars)
startDate / endDate	date-time	Yes	Project start and end dates
department	string	Yes	Department name (max 100 chars)
course	string	Yes	Course name (max 100 chars)
technologies	string	Yes	Comma-separated technology stack (max 255 chars)
isGroupProject	boolean	No	Whether this is a group project
groupMembers	string	No	Names of group members (max 500 chars)
isSynopsisSubmitted	boolean	No	Has the synopsis been submitted
projectDescription	string	No	Detailed description (max 1000 chars)
githubUrl / liveUrl	string	No	GitHub repository and live demo URLs
userId	integer	Yes	ID of the user submitting the project

7. E-Commerce API

A comprehensive e-commerce backend supporting hierarchical product categories, product management with images and properties, customer management, shopping cart, and order processing.

7.1 Endpoints — Categories

Method	Endpoint	Description
POST	/api/Ecommerce/CreateParentCategory	Create a top-level product category
GET	/api/Ecommerce/GetParentCategories	List parent categories for a user (userId)
DELETE	/api/Ecommerce/DeleteParentCategory	Delete a parent category by id
POST	/api/Ecommerce/CreateChildCategory	Create a sub-category under a parent
GET	/api/Ecommerce/GetChildCategories	List child categories for a user (userId)
DELETE	/api/Ecommerce/DeleteChildCategory	Delete a child category by id
POST	/api/Ecommerce/CreateCategoryWithChildren	Create parent and child categories in one call
GET	/api/Ecommerce/GetCategoryWithChildrenById	Get parent with its children by id

7.2 Endpoints — Products

Method	Endpoint	Description
GET	/api/Ecommerce/get-products	List all products for a user (userId)
GET	/api/Ecommerce/get-products-by-child-category	Filter products by child category
GET	/api/Ecommerce/get-products-by-parent-category/{parentCategoryId}	Filter by parent category
POST	/api/Ecommerce/CreateCompleteProduct	Create product with images and properties in one call
PUT	/api/Ecommerce/updateCompleteProduct	Update a complete product (id query param)
DELETE	/api/Ecommerce/delete-product/{id}	Delete a product by path ID
GET	/api/Ecommerce/get-product/{id}	Get a single product by path ID
POST	/api/Ecommerce/add-product-image	Add an image to a product
GET	/api/Ecommerce/get-product-images/{productId}	Get all images for a product
PUT	/api/Ecommerce/update-product-image/{id}	Update a product image
DELETE	/api/Ecommerce/delete-product-image/{id}	Delete a product image
POST	/api/Ecommerce/add-product-property	Add a custom property to a product
GET	/api/Ecommerce/get-product-properties/{productId}	Get all properties for a product
PUT	/api/Ecommerce/update-product-property/{id}	Update a product property
DELETE	/api/Ecommerce/delete-product-property/{id}	Delete a product property

7.3 Endpoints — Customers, Cart & Orders

Method	Endpoint	Description
POST	/api/Ecommerce/create-customer	Register a new customer
GET	/api/Ecommerce/get-customers	List all customers for a user (userId)
GET	/api/Ecommerce/get-customer/{id}	Get a single customer by path ID
PUT	/api/Ecommerce/update-customer/{id}	Update customer details
DELETE	/api/Ecommerce/delete-customer/{id}	Delete a customer
POST	/api/Ecommerce/create-address	Add a delivery address for a customer
GET	/api/Ecommerce/get-addresses/{custId}	Get all addresses for a customer
PUT	/api/Ecommerce/update-address/{addressId}	Update an address
DELETE	/api/Ecommerce/delete-address/{addressId}	Delete an address
POST	/api/Ecommerce/add-to-cart	Add an item to the shopping cart
GET	/api/Ecommerce/get-cart-by-customer/{custId}	Get cart items for a customer
PUT	/api/Ecommerce/update-cart-item/{cartId}	Update quantity/details of a cart item
DELETE	/api/Ecommerce/remove-cart-item/{cartId}	Remove a single item from cart
DELETE	/api/Ecommerce/clear-cart/{custId}	Clear all items from a customer's cart
POST	/api/Ecommerce/place-order/{custId}/{addressId}	Place order from cart for a customer
GET	/api/Ecommerce/get-orders	Get all orders for a user (userId)
GET	/api/Ecommerce/get-order-details/{orderId}	Get full details of an order
PUT	/api/Ecommerce/update-order/{orderId}	Update order status
DELETE	/api/Ecommerce/cancel-order/{orderId}	Cancel an order
GET	/api/Ecommerce/get-dashboard/{userId}	Get sales dashboard statistics

8. Employee App API

A lightweight employee management module providing CRUD operations for employees, with department and designation lookup support.

8.1 Endpoints

Method	Endpoint	Description
GET	/api/EmployeeApp/GetDepartments	Retrieve all departments
GET	/api/EmployeeApp/GetDesignationsByDeptId	Get designations for a department (deptId)
POST	/api/EmployeeApp/CreateEmployee	Create a new employee record
GET	/api/EmployeeApp/GetEmployees	List all employees
GET	/api/EmployeeApp/{id}	Get a single employee by path ID
PUT	/api/EmployeeApp/UpdateEmployee	Update employee details (id query param)
DELETE	/api/EmployeeApp/DeleteEmployee	Delete an employee (id query param)

8.2 Employee Model

Field	Type	Required	Description
fullName	string	Yes	Full name (max 150 chars)
email	string	Yes	Valid email address
phone	string	No	10-digit phone number
gender	string	No	Gender (max 6 chars: Male/Female/Other)
dateOfJoining	date-time	No	Employment start date
departmentId	integer	No	Foreign key to department
designationId	integer	No	Foreign key to designation
employeeType	string	No	e.g. Permanent, Contract, Part-time
salary	double	No	Monthly salary amount

9. Employee Onboarding API

A comprehensive HR onboarding module with full employee registration, master data management (departments, designations, cadre, etc.), company management, and advanced filtering.

9.1 Endpoints

Method	Endpoint	Description
GET	/api/EmployeeOnboarding/GetCompanies	List all companies
GET	/api/EmployeeOnboarding/GetCompanybyid	Get a company by id
POST	/api/EmployeeOnboarding/PostCompany	Create a new company
PUT	/api/EmployeeOnboarding/PutCompany	Update a company (id query param)
DELETE	/api/EmployeeOnboarding/DeleteCompany	Delete a company (id query param)
GET	/api/EmployeeOnboarding/getAllMaster	Get all master data records
GET	/api/EmployeeOnboarding/getDropdownMaster s	Get master data for dropdown population
GET	/api/EmployeeOnboarding/GetByMasterFor	Filter master data by category (masterFor + parentMasterId)
POST	/api/EmployeeOnboarding/CreateNewMaster	Create a new master data entry
POST	/api/EmployeeOnboarding/UpdateMaster	Update a master entry (id query param)
DELETE	/api/EmployeeOnboarding/DeleteMasterById	Delete a master entry by id
GET	/api/EmployeeOnboarding/GetEmployees	List all onboarded employees
POST	/api/EmployeeOnboarding/filterEmployees	Filter employees with advanced criteria
GET	/api/EmployeeOnboarding/GetEmployeeById	Get a single employee by id
POST	/api/EmployeeOnboarding/register	Register a full employee with family and experience data
PUT	/api/EmployeeOnboarding/UpdateEmployee	Update a full employee record
DELETE	/api/EmployeeOnboarding/{id}	Delete an employee by path ID

Note: The EmployeeRegistrationViewModel contains 70+ fields covering personal info, addresses, payroll, ESI/PF, bank details, family members, and work experience. Refer to the OpenAPI schema for the complete field list.

10. Enquiry Management API

Tracks customer enquiries from receipt through resolution. Supports filtering, categorization, and status progression workflows.

10.1 Endpoints

Method	Endpoint	Description
GET	/api/Enquiry/get-enquiries	Retrieve all enquiries
GET	/api/Enquiry/get-enquiry/{id}	Get a single enquiry by path ID
POST	/api/Enquiry/create-enquiry	Create a new enquiry
PUT	/api/Enquiry/update-enquiry/{id}	Update an existing enquiry
DELETE	/api/Enquiry/delete-enquiry/{id}	Delete an enquiry
POST	/api/Enquiry/filter-enquiries	Filter enquiries with pagination and sorting
GET	/api/Enquiry/get-categories	List all enquiry categories
POST	/api/Enquiry/create-category	Create an enquiry category
PUT	/api/Enquiry/update-category/{id}	Update a category
DELETE	/api/Enquiry/delete-category/{id}	Delete a category
GET	/api/Enquiry/get-statuses	List all enquiry statuses
POST	/api/Enquiry/create-status	Create a new status option
PUT	/api/Enquiry/update-status/{id}	Update a status
DELETE	/api/Enquiry/delete-status/{id}	Delete a status

10.2 EnquiryFilter Model (POST /filter-enquiries)

Field	Type	Required	Description
fromDate / toDate	date-time	No	Date range filter for enquiry date
categoryId	integer	No	Filter by category
statusId	integer	No	Filter by status
customerName	string	No	Partial name search
contactNo	string	No	Contact number search
sortBy	string	No	Field name to sort by
sortDir	string	No	asc or desc
page	integer	No	Page number (1–1000)
pageSize	integer	No	Results per page (1–200)

11. Fees Tracking API

Manages student fee enrollments for training/coaching institutes. Tracks total fees, EMI payments, outstanding balances, and provides batch-wise dashboard statistics.

11.1 Endpoints

Method	Endpoint	Description
GET	/api/FeesTracking/GetDashboardStats	Get summary statistics across all batches
GET	/api/FeesTracking/getAllEnrollments	List all student enrollments
GET	/api/FeesTracking/{id}	Get a single enrollment by path ID
POST	/api/FeesTracking/addNewEnrollment	Create a new student enrollment
PUT	/api/FeesTracking/updateEnrollment	Update enrollment details (id query param)
POST	/api/FeesTracking/FilterEnrollments	Filter enrollments by batch, status, or name search
DELETE	/api/FeesTracking/SoftDeleteById	Soft-delete an enrollment (marks as deleted)
DELETE	/api/FeesTracking/DeleteById	Hard delete an enrollment record
GET	/api/FeesTracking/batches	List all fee batches
POST	/api/FeesTracking/batches	Create a new batch
GET	/api/FeesTracking/batches/{id}	Get a single batch by path ID
PUT	/api/FeesTracking/batches/{id}	Update a batch
DELETE	/api/FeesTracking/batches/{id}	Delete a batch

11.2 StudentFee Model

Field	Type	Required	Description
studentName	string	Yes	Full name of the student (max 100 chars)
contactNo	string	Yes	10-digit mobile number (pattern validated)
totalFees	double	Yes	Total course fee amount (minimum 1)
emi1 / emi2 / emi3	double	No	Individual EMI payment amounts (minimum 0)
totalReceived	double	No	Sum of all payments received so far
status	string	No	Payment status (e.g. Paid, Pending, Partial)
batchId	integer	Yes	ID of the batch this student belongs to
isSoftDelete	boolean	No	Soft delete flag (set by system)

12. Goal Tracker API

A personal productivity application for setting goals, breaking them into milestones, managing recurring tasks, and tracking completion trends over time.

12.1 Endpoints — Users & Dashboard

Method	Endpoint	Description
POST	/api/GoalTracker/register	Register a new user account
POST	/api/GoalTracker/login	Authenticate a user
GET	/api/GoalTracker/getAllUsers	List all users
GET	/api/GoalTracker/getUserById	Get a user by id (query param)
PUT	/api/GoalTracker/updateUser	Update user profile (id query param)
DELETE	/api/GoalTracker/deleteUserById	Delete a user (id query param)
GET	/api/GoalTracker/dashboard	Get personal dashboard summary (userId)
GET	/api/GoalTracker/dashboardTaskbyDatefrequency	Get tasks by date and frequency
GET	/api/GoalTracker/trend	Get completion trend data (userId, frequency, date)
GET	/api/GoalTracker/recent-activity	Get recent activity log (userId)
POST	/api/GoalTracker/complete	Mark a task as completed

12.2 Endpoints — Goals & Tasks

Method	Endpoint	Description
GET	/api/GoalTracker/getAllGoalsByUser	List all goals for a user (userId)
GET	/api/GoalTracker/getGoal/{id}	Get a goal with its milestones by path ID
POST	/api/GoalTracker/createGoalWithMilestones	Create a goal with milestones in one call
PUT	/api/GoalTracker/updateGoalWithMilestones/{id}	Update goal and milestones
DELETE	/api/GoalTracker/deleteGoal/{id}	Delete a goal
GET	/api/GoalTracker/getAllTasks	List all tasks for a user (userId)
GET	/api/GoalTracker/getTask/{id}	Get a single task by path ID
POST	/api/GoalTracker/createTask	Create a new recurring task
PUT	/api/GoalTracker/updateTask/{id}	Update a task
DELETE	/api/GoalTracker/deleteTask/{id}	Delete a task
POST	/api/GoalTracker/createReminder	Create a reminder
GET	/api/GoalTracker/getReminders	Get reminders for a user (userId)
GET	/api/GoalTracker/getReminder/{id}	Get a single reminder
PUT	/api/GoalTracker/updateReminder/{id}	Update a reminder
DELETE	/api/GoalTracker/deleteReminder/{id}	Delete a reminder

12.3 TaskModel

Field	Type	Required	Description
taskName	string	Yes	Name of the task (max 255 chars)
description	string	No	Optional description
frequency	string	Yes	Recurrence: Daily, Weekly, or Monthly (pattern validated)
startDate / dueDate	date-time	No	Task start and due dates
isCompleted	boolean	No	Completion flag
userId	integer	Yes	Owner user ID

13. Leave Tracker API

Manages employee leave requests and balances. Employees request leave; managers review and approve. Tracks sick leave, paid leave, and custom leave type balances.

13.1 Endpoints

Method	Endpoint	Description
POST	/api/LeaveTracker/login	Employee or admin login
GET	/api/LeaveTracker/getAllEmployee	List all employees
GET	/api/LeaveTracker/getEmployeeById	Get an employee by id (query param)
POST	/api/LeaveTracker/CreateNewEmployee	Create a new employee record
PUT	/api/LeaveTracker/UpdateEmployee	Update employee details (id query param)
DELETE	/api/LeaveTracker/DeleteEmployee	Delete an employee (id query param)
GET	/api/LeaveTracker/GetAllBalances	Get all leave balances across all employees
GET	/api/LeaveTracker/GetBalanceByEmployee	Get leave balances for a specific employee (empId)
POST	/api/LeaveTracker/AddLeaveBalance	Add or adjust leave balance for an employee
DELETE	/api/LeaveTracker/deleteBalanceById	Delete a leave balance record by id
POST	/api/LeaveTracker/request	Submit a new leave request
GET	/api/LeaveTracker/request	Get all leave requests
GET	/api/LeaveTracker/GetLeaveRequestsbyEmpId	Get leave requests for an employee (empId)
GET	/api/LeaveTracker/dashboard/{empId}	Get leave dashboard for an employee

13.2 LeaveEmpRequest Model

Field	Type	Required	Description
empId	integer	Yes	Employee ID making the request
leaveDate	date-time	Yes	Date the request was submitted
fromDate	date-time	Yes	Start date of leave
toDate	date-time	Yes	End date of leave
reason	string	No	Reason for the leave request
leaveType	string	Yes	Type: SickLeave, PaidLeave, or custom type

14. Project Competition API

A platform for organizing coding/project competitions. Admins create competitions with start/end dates; participants register, submit projects, and admins approve, reject, or award winners.

14.1 Endpoints

Method	Endpoint	Description
GET	/api/ProjectCompetition/GetDashboardSummary	Get competition statistics dashboard
GET	/api/ProjectCompetition/GetAllCompetition	List all competitions
GET	/api/ProjectCompetition/competition/{id}	Get a single competition by path ID
POST	/api/ProjectCompetition/competition	Create a new competition
PUT	/api/ProjectCompetition/update/{id}	Update a competition
DELETE	/api/ProjectCompetition/delete/{id}	Delete a competition
GET	/api/ProjectCompetition/all-users	List all registered users
POST	/api/ProjectCompetition/register	Register a new user account
POST	/api/ProjectCompetition/login	User login
GET	/api/ProjectCompetition/user/{id}	Get a user by path ID
POST	/api/ProjectCompetition/project	Submit a project to a competition
GET	/api/ProjectCompetition/project/{id}	Get a project submission by path ID
PUT	/api/ProjectCompetition/update-submission/{id}	Update a project submission
DELETE	/api/ProjectCompetition/delete-submission/{id}	Delete a project submission
GET	/api/ProjectCompetition/project/byCompetition/{competitionId}	Get all submissions for a competition
PUT	/api/ProjectCompetition/project/approve/{id}	Approve a project submission
PUT	/api/ProjectCompetition/project/reject/{id}	Reject a project submission
PUT	/api/ProjectCompetition/project/winner/{id}	Set a project as winner with rank (rank query param, default 1)

15. Smart Parking API

A multi-tenant smart parking management system organized as: Client > Site > Building > Floor > Parking Spot. Supports vehicle entry/exit tracking, real-time availability, and admin reporting.

15.1 Endpoints — Users & Clients

Method	Endpoint	Description
POST	/api/SmartParking/register	Register a new user (with clientId)
POST	/api/SmartParking/login	User login
GET	/api/SmartParking/getAllUsersByClientId	Get all users for a client (clientId)
GET	/api/SmartParking/getUserById	Get a user by id (query param)
PUT	/api/SmartParking/updateUser	Update user profile
DELETE	/api/SmartParking/deleteUserById	Delete a user
GET	/api/SmartParking/GetAllClients	List all parking clients/operators
POST	/api/SmartParking/AddClient	Create a new parking client
POST	/api/SmartParking/UpdateClient	Update a parking client
POST	/api/SmartParking/DeleteClient	Delete a client (id query param)

15.2 Endpoints — Sites, Buildings & Floors

Method	Endpoint	Description
GET	/api/SmartParking/GetAllSites	Get all sites (clientId query param)
GET	/api/SmartParking/GetSiteById	Get a site by id
GET	/api/SmartParking/GetSitesByClientId	Get sites for a specific client
POST	/api/SmartParking/AddClientSite	Create a site with buildings
POST	/api/SmartParking/UpdateSite	Update a site
POST	/api/SmartParking/DeleteSite	Delete a site (id query param)
GET	/api/SmartParking/GetBuildingBySiteId	Get buildings for a site
GET	/api/SmartParking/GetFloorsBySiteId	Get floors for a site
GET	/api/SmartParking/GetFloorsByBuildingId	Get floors for a building
GET	/api/SmartParking/GetAllFloor	Get all floors
POST	/api/SmartParking/AddFloor	Create a new floor
POST	/api/SmartParking/UpdateFloor	Update a floor
POST	/api/SmartParking/DeleteFloor	Delete a floor (id query param)

15.3 Endpoints — Parking Operations

Method	Endpoint	Description
GET	/api/SmartParking/GetAllParking	Get all parking records
GET	/api/SmartParking/GetAllActiveParking	Get currently occupied spots

GET	/api/SmartParking/GetAllParkingByClientId	Parking records for a client
GET	/api/SmartParking/GetAllParkingByFloor	Parking records for a floor
GET	/api/SmartParking/GetAllParkingByBuilding	Parking records for a building
GET	/api/SmartParking/GetAllParkingBySite	Parking records for a site
POST	/api/SmartParking/AddParking	Record a vehicle entry (check-in)
POST	/api/SmartParking/MarkExit	Record a vehicle exit (check-out with extra charge)
POST	/api/SmartParking/DeleteParking	Delete a parking record (id query param)
POST	/api/SmartParking/GetAdminAllParkings	Advanced admin search across all dimensions

16. Survey API

A complete survey platform for creating surveys with multiple-choice questions, collecting anonymous or identified responses, and retrieving response analytics.

16.1 Endpoints

Method	Endpoint	Description
POST	/api/Survey/register	Register a survey creator account
POST	/api/Survey/login	User login
GET	/api/Survey/getAllUsers	List all users
GET	/api/Survey/getUserById	Get a user by id (query param)
PUT	/api/Survey/updateUser	Update user profile
DELETE	/api/Survey/deleteUserById	Delete a user
POST	/api/Survey/CreateSurveyWithQuestions	Create a complete survey with questions and options in one call
POST	/api/Survey/CreateSurvey	Create a survey (questions added separately)
GET	/api/Survey/GetAllSurveys	List all surveys
GET	/api/Survey/GetSurveysByUser/{userId}	Get surveys created by a user
POST	/api/Survey/AddQuestion	Add a question to an existing survey
GET	/api/Survey/GetQuestionsBySurveyId/{surveyId}	Get all questions for a survey
POST	/api/Survey/AddOption	Add an answer option to a question
GET	/api/Survey/GetOptionsByQuestionId/{questionId}	Get all options for a question
GET	/api/Survey/GetSurveyQuestionsWithOptions/{surveyId}	Get full survey with questions and options
POST	/api/Survey/SubmitSurvey	Submit a completed survey response
GET	/api/Survey/survey-response-by-survey/{surveyId}	Get all responses for a survey

16.2 CreateSurveyDto — One-Call Survey Creation

Field	Type	Required	Description
userId	integer	No	ID of the survey creator
surveyTitle	string	No	Title of the survey
surveyDescription	string	No	Brief description of the survey purpose
endDate	date-time	No	When the survey closes for responses
questions	CreateSurveyQuestionDto[]	No	Array of questions with options nested inside

17. User App API

A full-featured authentication and user management module supporting JWT access tokens, refresh tokens, paginated user search, OTP-based password reset, and password update workflows.

17.1 Endpoints

Method	Endpoint	Description
POST	/api/UserApp/CreateNewUser	Register a new user account
POST	/api/UserApp/login	Authenticate with email and password; returns JWT
POST	/api/UserApp/refresh	Refresh an expired JWT using a refresh token
GET	/api/UserApp/GetAllUsers	List all users (ApiResponse wrapper)
GET	/api/UserApp/searchUsers	Paginated user search (searchText, pageNumber, pageSize)
POST	/api/UserApp/send-reset-otp	Send OTP to email for password reset (emailId query param)
POST	/api/UserApp/verify-otp-reset-password	Verify OTP and set new password
PUT	/api/UserApp/update-password/{customerId}	Change password for authenticated user

17.2 JWT Refresh Model

Field	Type	Required	Description
emailId	string	No	User email address
token	string	No	Current (expired) access token
refreshToken	string	No	Valid refresh token received during login

18. Error Handling & HTTP Status Codes

All endpoints return HTTP 200 OK with a result wrapper. Use the result and message fields in ApiResponse to determine success or failure in your JavaScript code.

HTTP Code	Status	Description
200 OK	Success	Request processed. Check result field in ApiResponse body.
400 Bad Request	Validation Error	Request body failed model validation. Check required fields.
401 Unauthorized	Auth Required	Missing or invalid JWT token (UserApp module).
404 Not Found	Not Found	The requested resource ID does not exist.
500 Internal Server Error	Server Error	Unexpected backend error. Retry or contact administrator.

19. Integration Guide for HTML/JavaScript/Bootstrap

This section provides practical patterns for consuming the FreeProject API from your HTML/Bootstrap frontend using vanilla JavaScript fetch().

19.1 Base Configuration

```
const API_BASE = 'https://your-api-domain.com/api';
const HEADERS = { 'Content-Type': 'application/json' };

// Generic GET helper
async function apiGet(path) {
  const res = await fetch(API_BASE + path);
  return res.json();
}

// Generic POST helper
async function apiPost(path, body) {
  const res = await fetch(API_BASE + path, {
    method: 'POST', headers: HEADERS, body: JSON.stringify(body)
  });
  return res.json();
}
```

19.2 Example: Search Buses & Display with Bootstrap

```
async function searchBuses(from, to, date) {
  const data = await apiGet(
    `/BusBooking/searchBus?fromLocation=${from}&toLocation=${to}&travelDate=${date}`
  );
  const list = document.getElementById('busList');
  list.innerHTML = data.map(bus => `
    <div class='card mb-3'>
      <div class='card-body'>
        <h5 class='card-title'>${bus.busName}</h5>
        <p>Seats: ${bus.availableSeats}/${bus.totalSeats} | Price:
    ${bus.price}</p>
      </div></div>`).join('');
}
```

19.3 Example: Login with ApiResponse Handling

```
async function login(username, password) {
  const res = await apiPost('/BankLoan/login', { userName: username, password });
  if (res.result) {
    sessionStorage.setItem('user', JSON.stringify(res.data));
    showAlert('success', 'Login successful!');
  } else {
    showAlert('danger', res.message || 'Login failed!');
  }
}
```

```
Bootstrap Alert Helper: function showAlert(type, msg) { document.getElementById('alert').innerHTML = `<div class='alert alert-${type}'>${msg}</div>`; }
```

19.4 Quick Reference — Common Query Parameters

Parameter	Type	Used By
userId / customerId	integer	Most modules to scope data to a specific user
id	integer	Generic record identifier in path or query
vendorId	integer	BusBooking — scope to a specific vendor
clientId	integer	SmartParking — scope to a parking operator
emplId / bankEmployeeId	integer	LeaveTracker, BankLoan — employee scoping
status	string	Filter by status string across multiple modules
pageNumber / pageSize	integer	UserApp searchUsers for pagination
fromDate / toDate	date-time	Date range filters in Enquiry, SmartParking

FreeProject API Documentation | Version 1.0 | Internet Technology Final Coursework
 Built with HTML • JavaScript • Bootstrap 5 | Backend: ASP.NET Core | OpenAPI 3.0.1